

6.5840 Lab 5: Sharded Key/Value Service

[Collaboration policy](#) // [Submit lab](#) // [Setup Go](#) // [Guidance](#) // [Piazza](#)

Introduction

You can either do a [final project](#) based on your own ideas, or this lab.

In this lab you'll build a key/value storage system that "shards," or partitions, the keys over a set of Raft-replicated key/value server groups (shardgrps). A shard is a subset of the key/value pairs; for example, all the keys starting with "a" might be one shard, all the keys starting with "b" another, etc. The reason for sharding is performance. Each shardgrp handles puts and gets for just a few of the shards, and the shardgrps operate in parallel; thus total system throughput (puts and gets per unit time) increases in proportion to the number of shardgrps.

[image]

The sharded key/value service has the components shown above. Shardgrps (shown with blue squares) store shards with keys: shardgrp 1 holds a shard storing key "a", and shardgrp 2 holds a shard storing key "b". Clients of the sharded key/value service interact with the service through a clerk (shown with a green circle), which implements Get and Put methods. To find the shardgrp for a key passed to Put/Get, the clerk gets the configuration from the kvsrv (shown with a black square), which you implemented in Lab 2. The configuration (not shown) describes the mapping from shards to shardgrps (e.g., shard 1 is served by shardgrp 3).

An administrator (i.e., the tester) uses another client, the controller (shown with a purple circle), to add/remove shardgrps from the cluster and update which shardgrp should serve a shard. The controller has one main method: `ChangeConfigTo`, which takes as argument a new configuration and changes the system from the current configuration to the new configuration; this involves moving shards to new shardgrps that are joining the system and moving shards away from shardgrps that are leaving the system. To do so the controller 1) makes RPCs (`FreezeShard`, `InstallShard`, and `DeleteShard`) to shardgrps, and 2) updates the configuration stored in kvsrv.

The reason for the controller is that a sharded storage system must be able to shift shards among shardgrps. One reason is that some shardgrps may become more loaded than others, so that shards need to be moved to balance the load. Another reason is that shardgrps may join and leave the system: new shardgrps may be added to increase capacity, or existing shardgrps may be taken offline for repair or retirement.

The main challenges in this lab will be ensuring linearizability of Get/Put operations while handling 1) changes in the assignment of shards to shardgrps, and 2) recovering from a controller that fails or is partitioned during `ChangeConfigTo`.

1. `ChangeConfigTo` moves shards from one shardgrp to another. A risk is that some clients might use the old shardgrp while other clients use the new shardgrp, which could break linearizability. You will need to ensure that at most one shardgrp is serving requests for each shard at any one time.

2. If `ChangeConfigTo` fails while reconfiguring, some shards may be inaccessible if they have started but not completed moving from one shardgrp to another. To make forward progress, the tester starts a new controller, and your job is to ensure that the new one completes the reconfiguration that the previous controller started.

This lab uses "configuration" to refer to the assignment of shards to shardgrps. This is not the same as Raft cluster membership changes. You don't have to implement Raft cluster membership changes.

A shardgrp server is a member of only a single shardgrp. The set of servers in a given shardgrp will never change.

Only RPC may be used for interaction among clients and servers. For example, different instances of your server are not allowed to share Go variables or files.

In Part A, you will implement a working `shardctrler`, which will store and retrieve configurations in a `kvsrv`. You will also implement the `shardgrp`, replicated with your Raft `rsm` package, and a corresponding `shardgrp` clerk. The `shardctrler` talks to the `shardgrp` clerks to move shards between different groups.

In Part B, you will modify your `shardctrler` to handle failures and partitions during config changes. In Part C, you will extend your `shardctrler` to allow for concurrent controllers without interfering with each other. Finally, in Part D, you will have the opportunity to extend your solution in any way you like.

This lab's sharded key/value service follows the same general design as Flat Datacenter Storage, BigTable, Spanner, FAWN, Apache HBase, Rosebud, Spinnaker, and many others. These systems differ in many details from this lab, though, and are also typically more sophisticated and capable. For example, the lab doesn't evolve the sets of peers in each Raft group; its data and query models are simple; and so on.

Lab 5 will use your `kvsrv` from Lab 2, and your `rsm` and Raft from Lab 4. Your Lab 5 and Lab 4 must use the same `rsm` and Raft implementations.

You may use late hours for Part A, but you may not use late hours for Parts B-D.

Getting Started

Do a git pull to get the latest lab software.

We supply you with tests and skeleton code in `src/shardkv1`:

- `client.go` for the `shardkv` clerk
- `shardcfg` package for computing shard configurations
- `shardgrp` package: for the `shardgrp` clerk and server.
- `shardctrler` package, which contains `shardctrler.go` with methods for the controller to change a configuration (`ChangeConfigTo`) and to get a configuration (`Query`)

To get up and running, execute the following commands:

```
$ cd ~/6.5840
$ git pull
...
$ cd src
$ make RUN="-run 5A" shardkv
go build -race -o main/kvsrvld main/kvsrvld.go
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run 5A
=== RUN    TestInitQuery5A
Test (5A): Init and Query ... (reliable network)...
...
```

Part A: Moving shards

Your first job is to implement `shardgrps` and the `InitConfig`, `Query`, and `ChangeConfigTo` methods when there are no failures. We have given you the code for describing a configuration, in `shardkv1/shardcfg`. Each `shardcfg.ShardConfig` has a unique identifying number, a mapping from shard number to group number, and a mapping from group number to the list of servers replicating that group. There will usually be more shards than groups (so that each group serves more than one shard), in order that load can be shifted at a fairly fine granularity.

Implement these two methods in `shardctrler/shardctrler.go`:

- The `InitConfig` method receives the first configuration, passed to it by the tester as a `shardcfg.ShardConfig`. `InitConfig` should store the configuration in an instance of Lab 2's `kvsrv`.
- The `Query` method returns the current configuration; it should read the configuration from `kvsrv`, previously stored there by `InitConfig`.

Implement `InitConfig` and `Query`, and store the configuration in `kvsrv`. You're done when your code passes the first test. Note this task doesn't require any `shardgrps`.

```
$ cd ~/6.5840/src
$ make RUN="-run TestInitQuery5A" shardkv
go build -race -o main/kvsrvld main/kvsrvld.go
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run TestInitQuery5A
=== RUN    TestInitQuery5A
Test (5A): Init and Query ... (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs      3 #Ops      0
--- PASS: TestInitQuery5A (0.13s)
PASS
ok      6.5840/shardkv1 1.143s
```

- Implement `InitConfig` and `Query` by storing and reading the initial configuration from `kvsrv`: use the `Get/Put` methods of `ShardCtrler.IKVClerk` to talk to `kvsrv`, use the `String` method of `ShardConfig` to turn a `ShardConfig` into a string that you can pass to `Put`, and use the `shardcfg.FromString()` function to turn a string into a `ShardConfig`.

Implement an initial version of `shardgrp` in `shardkv1/shardgrp/server.go` and a corresponding clerk in `shardkv1/shardgrp/client.go` by copying code from your Lab 4 `kvraft` solution.

Implement a clerk in `shardkv1/client.go` that uses the `Query` method to find the `shardgrp` for a key, and then talks to that `shardgrp`. You're done when your code passes the Static test.

```
$ cd ~/6.5840/src
$ make RUN="-run TestStaticOneShardGroup5A" shardkv
go build -race -o main/kvsrvld main/kvsrvld.go
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run TestStaticOneShardGroup5A
=== RUN    TestStaticOneShardGroup5A
Test (5A): one shard group ... (reliable network)...
... Passed -- time 9.4s #peers 1 #RPCs 822 #Ops 420
--- PASS: TestStaticOneShardGroup5A (9.52s)
PASS
ok      6.5840/shardkv1 10.542s
```

- Copy code from your `kvraft` `client.go` and `server.go` for `Put` and `Get`, and any other code you need from `kvraft`.
- The code in `shardkv1/client.go` provides the `Put/Get` clerk for the overall system: it finds out which `shardgrp` holds the desired key's shard by invoking the `Query` method, and then talks to the `shardgrp` that holds that shard.
- Implement `shardkv1/client.go`, including its `Put/Get` methods. Use `shardcfg.Key2Shard()` to find the shard number for a key. The tester passes a `ShardCtrler` object to `MakeClerk` in `shardkv1/client.go`. Retrieve the current configuration using the `Query` method.
- To `put/get` a key from a `shardgrp`, the `shardkv` clerk should create a `shardgrp` clerk for the `shardgrp` by calling `shardgrp.MakeClerk`, passing in the servers found in the configuration and the `shardkv` clerk's `ck.clnr`. Use the `GidServers()` method from `ShardConfig` to get the group for a shard.
- `shardkv1/client.go`'s `Put` must return `ErrMaybe` when the reply was maybe lost, but this `Put` invokes `shardgrp`'s `Put` to talk a particular `shardgrp`. The inner `Put` can signal this with an error.
- Upon creation, the first `shardgrp` (`shardcfg.Gid1`) should initialize itself to own all shards.

Now you should support movement of shards among groups by implementing the `ChangeConfigTo` method, which changes from an old configuration to a new configuration. The new configuration may include new `shardgrps` that are not present in the old configuration, and may exclude `shardgrps` that were present in the old configuration. The controller should move shards (the key/value data) so that the set of shards stored by each `shardgrp` matches the new configuration.

The approach we suggest for moving a shard is for `ChangeConfigTo` to first "freeze" the shard at the source `shardgrp`, causing that `shardgrp` to reject `Put`'s for keys in the moving shard. Then, copy (install) the shard to the destination `shardgrp`; then delete the frozen shard. Finally, post a new configuration so that clients can find the moved shard. A nice property of this approach is that it avoids any direct interactions among the `shardgrps`. It also supports serving shards that are not affected by an ongoing configuration change.

To be able to order changes to the configuration, each configuration has a unique number `Num` (see `shardcfg/shardcfg.go`). The tester in Part A invokes `ChangeConfigTo` sequentially, and the configuration passed to `ChangeConfigTo` will have a `Num` one larger than the previous one; thus, a configuration with a higher `Num` is newer than one with a lower `Num`.

The network may delay `RPCs`, and `RPCs` may arrive out of order at the `shardgrps`. To reject old

FreezeShard, InstallShard, and DeleteShard RPCs, they should include Num (see shardgrp/shardrpc/shardrpc.go), and shardgrps must remember the largest Num they have seen for each shard.

Implement ChangeConfigTo (in shardctrler/shardctrler.go) and extend shardgrp to support freeze, install, and delete. ChangeConfigTo should always succeed in Part A because the tester doesn't induce failures in this part. You will need to implement FreezeShard, InstallShard, and DeleteShard in shardgrp/client.go and shardgrp/server.go using the RPCs in the shardgrp/shardrpc package, and reject old RPCs based on Num. You will also need modify the shardkv clerk in shardkv1/client.go to handle ErrWrongGroup, which a shardgrp should return if it isn't responsible for the shard.

You have completed this task when you pass the JoinBasic and DeleteBasic tests. These tests focus on adding shardgrps; you don't have to worry about shardgrps leaving just yet.

- A shardgrp should respond with an ErrWrongGroup error to a client Put/Get with a key that the shardgrp isn't responsible for (i.e., for a key whose shard is not assigned to the shardgrp). You will have to modify shardkv1/client.go to reread the configuration and retry the Put/Get.
- Note that you will have to run FreezeShard, InstallShard, and DeleteShard through your rsm package, just like Put and Get.
- You can send an entire map as your state in an RPC request or reply, which may help keep the code for shard transfer simple.
- If one of your RPC handlers includes in its reply a map (e.g. a key/value map) that's part of your server's state, you may get bugs due to races. The RPC system has to read the map in order to send it to the caller, but it isn't holding a lock that covers the map. Your server, however, may proceed to modify the same map while the RPC system is reading it. The solution is for the RPC handler to include a copy of the map in the reply.

Extend ChangeConfigTo to handle shard groups that leave; i.e., shardgrps that are present in the current configuration but not in the new one. Your solution should pass TestJoinLeaveBasic5A now. (You may have handled this scenario already in the previous task, but the previous tests didn't test for shardgrps leaving.)

Make your solution pass all Part A tests, which check that your sharded key/value service supports many groups joining and leaving, shardgrps restarting from snapshots, processing Gets while some shards are offline or involved in a configuration change, and linearizability when many clients interact with the service while the tester concurrently invokes the controller's ChangeConfigTo to rebalance shards.

```
$ cd ~/6.5840/src
$ make RUN="-run 5A" shardkv
go build -race -o main/kvsrv1d main/kvsrv1d.go
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run 5A
Test (5A): Init and Query ... (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 3 #Ops 0
Test (5A): one shard group ... (reliable network)...
... Passed -- time 5.1s #peers 1 #RPCs 792 #Ops 180
Test (5A): a group joins... (reliable network)...
... Passed -- time 12.9s #peers 1 #RPCs 6300 #Ops 180
Test (5A): delete ... (reliable network)...
... Passed -- time 8.4s #peers 1 #RPCs 1533 #Ops 360
Test (5A): basic groups join/leave ... (reliable network)...
... Passed -- time 13.7s #peers 1 #RPCs 5676 #Ops 240
Test (5A): many groups join/leave ... (reliable network)...
```

```

... Passed -- time 22.1s #peers 1 #RPCs 3529 #Ops 180
Test (5A): many groups join/leave ... (unreliable network)...
... Passed -- time 54.8s #peers 1 #RPCs 5055 #Ops 180
Test (5A): shutdown ... (reliable network)...
... Passed -- time 11.7s #peers 1 #RPCs 2807 #Ops 180
Test (5A): progress ... (reliable network)...
... Passed -- time 8.8s #peers 1 #RPCs 974 #Ops 82
Test (5A): progress ... (reliable network)...
... Passed -- time 13.9s #peers 1 #RPCs 2443 #Ops 390
Test (5A): one concurrent clerk reliable... (reliable network)...
... Passed -- time 20.0s #peers 1 #RPCs 5326 #Ops 1248
Test (5A): many concurrent clerks reliable... (reliable network)...
... Passed -- time 20.4s #peers 1 #RPCs 21688 #Ops 10500
Test (5A): one concurrent clerk unreliable ... (unreliable network)...
... Passed -- time 25.8s #peers 1 #RPCs 2654 #Ops 176
Test (5A): many concurrent clerks unreliable... (unreliable network)...
... Passed -- time 25.3s #peers 1 #RPCs 7553 #Ops 1896
PASS
ok      6.5840/shardkv1 243.115s
$

```

Your solution must continue serving shards that are not affected by an ongoing configuration change.

Part B: Handling a failed controller

The controller is a short-lived command, which an administrator invokes: it moves shards and then exits. But, it may fail or lose network connectivity while moving shards. The main task in this part of the lab is recovering from a controller that fails to complete `ChangeConfigTo`. The tester starts a new controller and invokes its `ChangeConfigTo` after partitioning the first controller; you have to modify the controller so that the new one finishes the reconfiguration. The tester calls `InitController` when starting a controller; you can modify that function to check whether an interrupted configuration change needs to be completed.

A good approach to allowing a controller to finish a reconfiguration that a previous one started is to keep two configurations: a current one and a next one, both stored in the controller's `kvsrv`. When a controller starts a reconfiguration, it stores the next configuration. Once a controller completes the reconfiguration, it makes the next configuration the current one. Modify `InitController` to first check if there is a stored next configuration with a higher configuration number than the current one, and if so, complete the shard moves necessary to reconfigure to the next one.

Modify `shardctrler` to implement the above approach. A controller that picks up the work from a failed controller may repeat `FreezeShard`, `InstallShard`, and `Delete` RPCs; `shardgrps` can use `Num` to detect duplicates and reject them. You have completed this task if your solution passes the Part B tests.

```

$ cd ~/6.5840/src
$ make RUN="-run 5B" shardkv
go build -race -o main/kvsrv1d main/k
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run 5B
Test (5B): Join/leave while a shardgrp is down... (reliable network)...
... Passed -- time 9.2s #peers 1 #RPCs 899 #Ops 120
Test (5B): recover controller ... (reliable network)...
... Passed -- time 26.4s #peers 1 #RPCs 3724 #Ops 360

```

```
PASS
ok      6.5840/shardkv1 35.805s
$
```

- The tester calls `InitController` when starting a controller; you can implement recovery in that method in `shardctrler/shardctrler.go`.

Part C: Concurrent configuration changes

In this part of the lab you will modify the controller to allow for concurrent controllers. When a controller crashes or is partitioned, the tester will start a new controller, which must finish any work that the old controller might have in progress (i.e., finishing moving shards like in Part B). This means that several controllers may run concurrently and send RPCs to the `shardgrps` and the `kvsrv` that stores configurations.

The main challenge is to ensure these controllers don't step on each other. In Part A you already fenced all the `shardgrp` RPCs with `Num` so that old RPCs are rejected. Even if several controllers pick up the work of an old controller concurrently, one of them succeeds and the others repeat all the RPCs, the `shardgrps` will ignore them.

Thus the challenging case left is to ensure that only one controller updates the next configuration to avoid that two controllers (e.g., a partitioned one and a new one) put different configurations in the next one. To stress this scenario, the tester runs several controllers concurrently and each one computes the next configuration by reading the current configuration and updating it for a `shardgrp` that left or joined, and then the tester invokes `ChangeConfigTo`; thus multiple controllers may invoke `ChangeConfigTo` with different configuration with the same `Num`. You can use the version number of a key and versioned Puts to ensure that only one controller updates the next configuration and that the other invocations return without doing anything.

Modify your controller so that only one controller can post a next configuration for a configuration `Num`. The tester will start many controllers but only one should start `ChangeConfigTo` for a new configuration. You have completed this task if you pass the concurrent tests of Part C:

```
$ cd ~/6.5840/src
$ make RUN="-run TestConcurrentReliable5C" shardkv
go build -race -o main/kvsrv1d main/k
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run TestConcurrentReliable5C
Test (5C): Concurrent ctrlers ... (reliable network)...
... Passed -- time 8.2s #peers 1 #RPCs 1753 #Ops 120
PASS
ok      6.5840/shardkv1 8.364s
$ make RUN="-run TestAcquireLockConcurrentUnreliable5C" shardkv
go build -race -o main/kvsrv1d main/k
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run TestAcquireLockConcurrentUnreliable5C
Test (5C): Concurrent ctrlers ... (unreliable network)...
... Passed -- time 23.8s #peers 1 #RPCs 1850 #Ops 120
PASS
ok      6.5840/shardkv1 24.008s
$
```

- See `concurCtrler` in `test.go` to see how the tester runs controllers concurrently.

In this exercise you will put recovery of an old controller together with a new controller: a new controller should perform recovery from Part B. If the old controller was partitioned during `ChangeConfigTo`, you will have to make sure that the old controller doesn't interfere with the new controller. If all the controller's updates are already properly fenced with Num checks from Part B, you don't have to write extra code. You have completed this task if you pass the Partition tests.

```
$ cd ~/6.5840/src
$ make RUN="-run Partition" shardkv
go build -race -o main/kvsrv1d main/k
go build -race -o main/shardgrp1d main/shardgrp1d.go
cd shardkv1; go test -timeout 15m -v -race -run Partition
Test (5C): partition controller in join... (reliable network)...
... Passed -- time 7.8s #peers 1 #RPCs 876 #Ops 120
Test (5C): controllers with leased leadership ... (reliable network)...
... Passed -- time 36.8s #peers 1 #RPCs 3981 #Ops 360
Test (5C): controllers with leased leadership ... (unreliable network)...
... Passed -- time 52.4s #peers 1 #RPCs 2901 #Ops 240
Test (5C): controllers with leased leadership ... (reliable network)...
... Passed -- time 60.2s #peers 1 #RPCs 27415 #Ops 11182
Test (5C): controllers with leased leadership ... (unreliable network)...
... Passed -- time 60.5s #peers 1 #RPCs 11422 #Ops 2336
PASS
ok      6.5840/shardkv1 217.779s
$
```

You have completed implementing a highly-available sharded key/value service with many shard groups for scalability, reconfiguration to handle changes in load, and with a fault-tolerant controller; congrats!

Rerun all tests to check that your recent changes to the controller haven't broken earlier tests.

Gradescope will rerun the Lab 3A-D and Lab 4A-C tests on your submission, in addition to the 5C tests. Before submitting, double check that your solution works:

```
$ make raft1
$ make kvraft1
$ make shardkv
```

Part D: Extend your solution

In this final part of the lab, you get to extend your solution in any way you like. You will have to write your own tests for whatever extensions you choose to implement.

Implement one of the ideas below or come up with your own idea. Write a paragraph in a file `extension.md` describing your extension, and upload `extension.md` to Gradescope. If you would like to do one of the harder, open-ended extensions, feel free to partner up with another student in the class.

Here are some ideas for possible extension (the first few are easy and the later ones are more open ended):

- Change the tester to use `kvraft` instead of `kvsrv` (i.e., replace the `kvsrv.StartKVServer` in `MakeTestMaxRaft` in `test.go` with `kvraft.StartKVServer`) so that the controller uses your `kvraft` to store

its configuration. Write a test that checks that the controller can query and update the configuration while one of the kvraft peers is down. The existing code for the tester is distributed across `src/kvtest1`, `src/shardkv1`, and `src/tester1`.

- Change `kvsrv` to implement exactly-once semantics for Put/Get as in [Lab 2](#) from last year (see the dropped messages part). You may be able to port over some tests from 2024 instead of having to write your own from scratch. Implement exactly-once also in your `kvraft`.
- Change `kvsrv` to support a `Range` function, which returns all keys in the range low key to high key. The lazy way to implement `Range` is to iterate through the key/value map that the server maintains; a better way is to use a data structure that supports range searches (e.g., B-tree). Include a test that fails the lazy solution but passes on the better solution.
- Modify your `kvraft` implementation to allow the leader to serve Gets without running the Get through `rsm`. That is, implement the optimization described at the end of section 8 of the raft paper, including leases, to ensure that `kvraft` maintains linearizability. Your implementation should pass the existing `kvraft` tests. You should also add a test that checks that your optimized implementation is faster (e.g., by comparing the number of RPCs) and a test that checks that term switches are slower because a new leader must wait until the lease has expired.
- Support transactions in `kvraft` so that a developer can perform several Put and Gets atomically. Once you have transaction, you don't need versioned Puts anymore; transactions subsume versioned Puts. Look at [etcd's transactions](#) for an example interface. Write tests to demonstrate your extension works.
- Modify `shardkv` to support transactions so that a developer can perform several Puts and Gets atomically across shards. This requires implementing two-phase-commit and two-phase locking. Write tests to demonstrate that your extension works.

Handin procedure

Before submitting, please run *all* the tests one final time.

Before submitting, double check that your solution works with:

```
$ make raft1
$ make kvraft1
$ make shardkv
```

